

DSA Revision Notes

Topic 4 — Binary Search & All Variations

Classical BS · Lower/Upper Bound · Rotated Arrays · 2D Matrix Search
Search on Answer · Fractional BS · Peak Finding · 15+ C++ examples

10

Sections

20+

C++ Examples

8

Visualizations

25+

LeetCode Problems

TOC Table of Contents**1. Introduction & Core Intuition**

→ Why $O(\log n)$? · Convergence proof · When to use

2. Template 1 — Classical Binary Search

→ Exact match · $lo \leq hi$ variant · Detailed step trace

3. Template 2 — Lower Bound & Upper Bound

→ First occurrence · Last occurrence · Count occurrences
→ `std::lower_bound` / `upper_bound` from STL

4. Template 3 — Left-biased ($lo < hi$) Template

→ Finding leftmost valid position · Minimise answer

5. Rotated Sorted Arrays

→ Search in Rotated Array · Find Min in Rotated Array
→ With duplicates — edge cases

6. 2D Matrix Binary Search

→ Search a 2D Matrix (treat as 1D) · Search Matrix II

7. Binary Search on Answer (Parametric Search)

→ Concept · Feasibility function · Minimise/Maximise
→ Koko Eating Bananas · Capacity to Ship · Minimised Max
→ Split Array Largest Sum · Minimum Days to Bloom

8. Peak Finding

→ Find Peak Element · Mountain Array · Bitonic Search

9. Advanced Variations

→ Fractional/Real-valued Binary Search
→ Median of Two Sorted Arrays · $\text{Sqrt}(x)$
→ Find K-th Smallest in Sorted Matrix

10. Complexity Cheat Sheet & Decision Tree

1 Introduction & Core Intuition

1.1 What is Binary Search?

Binary Search is a **divide-and-conquer** algorithm that locates a target in a **sorted** search space in $O(\log n)$ time. At each step, we compare the target with the **middle element** and eliminate half the remaining candidates. The key invariant: **the answer always lies within [lo, hi]**.

Step-by-step trace: search for 14 in sorted array



1.2 Why $O(\log n)$?

After each comparison, the search space is **halved**. Starting with n elements: after 1 step $\rightarrow n/2$, after 2 $\rightarrow n/4$, ..., after k steps $\rightarrow n/2^k$. We stop when 1 element remains: $n/2^k = 1 \rightarrow k = \log_2(n)$. For $n = 1,000,000,000$ (1 billion), binary search needs only **30 comparisons**.

Search space shrinking — target=14, array size=10

Step	Search Space	Action
		arr[4]=9 < 14 → search right half
		arr[7]=17 > 14 → search left half
		arr[5]=11 < 14 → search right
		arr[6]=14 == target → FOUND

■ Core Requirement: Monotone Predicate

Binary Search requires a MONOTONE predicate: the search space must be divided into two contiguous halves where one half satisfies the condition and the other does not. It does NOT require the data to be a literal sorted array — just a monotone property. Example: 'can we ship all packages in D days?' is monotone in D (once true, stays true).

1.3 The Three Templates

Template	Condition	Terminates When	Best For
T1: lo <= hi	Compare with mid exactly	lo > hi (no overlap)	Exact target search
T2: lo < hi	Post-process needed	lo == hi (1 element)	Lower/Upper bound, min/max answer
T3: lo+1 < hi	Always 2+ elements in loop	lo+1 == hi (2 elements)	Peak finding, first/last TRUE

Table 1: The three binary search templates and when to use each

2.2 LC 704 — Binary Search (Easy)

Direct application of Template 1 on a sorted array.

```
C++
1  int search(vector<int>& nums, int target) {
2      int lo = 0, hi = nums.size() - 1;
3      while (lo <= hi) {
4          int mid = lo + (hi - lo) / 2;
5          if (nums[mid] == target) return mid;
6          else if (nums[mid] < target) lo = mid + 1;
7          else hi = mid - 1;
8      }
9      return -1;
10 }
```

2.3 Complete Invariant Analysis

Understanding why the loop terminates correctly and doesn't infinite-loop is critical.

Scenario	Condition	Action	Why Safe
arr[mid] == target	lo <= hi	Return mid	Found, done
arr[mid] < target	lo <= hi	lo = mid+1	mid is excluded — not the answer
arr[mid] > target	lo <= hi	hi = mid-1	mid is excluded — not the answer
lo > hi	—	Return -1	Search space exhausted, not present
lo == hi	lo <= hi (true)	Check arr[mid]	Single element: check and terminate
lo == hi-1	lo <= hi (true)	Check mid=(lo+hi)/2=lo	Two elements: one step determines

Table 2: Exhaustive invariant analysis for Template 1

3 Template 2 — Lower Bound & Upper Bound

3.1 Lower Bound — First Position $\geq$ target

Lower bound: find the leftmost index where $\text{arr}[i] \geq \text{target}$. If all elements are smaller, returns n (one past end). Key change from Template 1: when $\text{arr}[\text{mid}] \geq \text{target}$, we **don't discard mid** — it could be our answer. So $\text{hi} = \text{mid}$ (not $\text{mid}-1$). Loop: $\text{lo} < \text{hi}$.

```
C++
1 // Lower bound: first index where arr[i] >= target
2 int lowerBound(vector<int>& arr, int target) {
3     int lo = 0, hi = arr.size(); // hi = n (past-end sentinel)
4     while (lo < hi) { // NOTE: strict '<' not '<='
5         int mid = lo + (hi - lo) / 2;
6         if (arr[mid] < target) {
7             lo = mid + 1; // mid is too small, exclude it
8         } else {
9             hi = mid; // mid could be the answer, keep it
10        }
11    }
12    return lo; // lo == hi when loop ends - the answer
13 }
14 // Returns n if all elements < target (no valid index)
15 // Time: O(log n) Space: O(1)
16
17 // Example: arr=[1,2,4,4,5,6], target=4
18 // Returns index 2 (first occurrence of 4)
```

3.2 Upper Bound — First Position $>$ target

Upper bound: find the leftmost index where $\text{arr}[i] > \text{target}$. Combined with lower bound: $\text{count}(\text{target}) = \text{upper_bound} - \text{lower_bound}$.

C++

```
1  vector<int> searchRange(vector<int>& nums, int target) {
2      int n = nums.size();
3
4      // First occurrence = lower_bound
5      int lo = 0, hi = n;
6      while (lo < hi) {
7          int mid = lo + (hi - lo) / 2;
8          if (nums[mid] < target) lo = mid + 1;
9          else hi = mid;
10     }
11     int first = lo;
12     if (first == n || nums[first] != target) return {-1, -1};
13
14     // Last occurrence = upper_bound - 1
15     lo = first; hi = n;
16     while (lo < hi) {
17         int mid = lo + (hi - lo) / 2;
18         if (nums[mid] <= target) lo = mid + 1;
19         else hi = mid;
20     }
21     int last = lo - 1;
22
23     return {first, last};
24 }
25 // Time: O(log n)   Space: O(1)
```

3.4 STL lower_bound / upper_bound

C++

```

1  #include <algorithm>
2  #include <vector>
3  using namespace std;
4
5  vector<int> v = {1, 2, 4, 4, 5, 6};
6
7  // lower_bound: iterator to first element >= target
8  auto it1 = lower_bound(v.begin(), v.end(), 4);
9  int idx1 = it1 - v.begin();    // idx1 = 2
10
11 // upper_bound: iterator to first element > target
12 auto it2 = upper_bound(v.begin(), v.end(), 4);
13 int idx2 = it2 - v.begin();    // idx2 = 4
14
15 // Count occurrences
16 int cnt = idx2 - idx1;        // cnt = 2
17
18 // binary_search: returns bool (exists or not)
19 bool found = binary_search(v.begin(), v.end(), 4); // true
20
21 // Custom comparator (e.g., search on pairs by first element)
22 vector<pair<int,int>> vp = {{1,10},{3,20},{5,30}};
23 auto it3 = lower_bound(vp.begin(), vp.end(), make_pair(3, INT_MIN));
24 // finds first pair with first >= 3
25
26 // All STL variants: O(log n) for random access iterators

```

Function	Returns	Found if...	Not Found
<code>lower_bound(b,e,x)</code>	Iterator to first elem $\geq x$	$*it == x$	$it == end$ or $*it != x$
<code>upper_bound(b,e,x)</code>	Iterator to first elem $> x$	$(it-1)$ points to x	Always valid iterator
<code>binary_search(b,e,x)</code>	bool (true/false)	returns true	returns false
<code>equal_range(b,e,x)</code>	pair of $\{lb, ub\}$	$second - first > 0$	$second == first$

Table 3: STL binary search functions comparison

■ ■ Bias Rules — Prevent Infinite Loop

LEFT-biased $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$ → rounds DOWN → use when $\text{hi} = \text{mid}$ (keep left side)

RIGHT-biased $\text{mid} = \text{lo} + (\text{hi} - \text{lo} + 1) / 2$ → rounds UP → use when $\text{lo} = \text{mid}$ (keep right side)

Choosing wrong bias causes infinite loop when $\text{lo} + 1 == \text{hi}$. Always pair correctly:

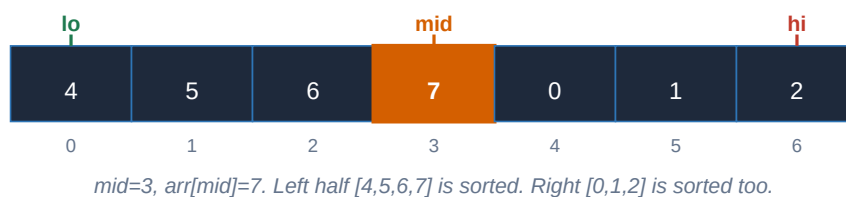
If you write $\text{hi} = \text{mid}$ → use LEFT bias $(\text{lo} + (\text{hi} - \text{lo}) / 2)$

If you write $\text{lo} = \text{mid}$ → use RIGHT bias $(\text{lo} + (\text{hi} - \text{lo} + 1) / 2)$

5 Rotated Sorted Arrays

5.1 Core Insight for Rotated Arrays

A rotated sorted array like $[4, 5, 6, 7, 0, 1, 2]$ has a key property: **splitting at any mid, at least one half is always fully sorted**. We determine which half is sorted, check if target lies in it, and recurse accordingly. This lets us maintain $O(\log n)$ even without knowing the rotation point.



5.2 Search in Rotated Sorted Array (LC 33) — Medium

C++

```
1  int search(vector<int>& nums, int target) {
2      int lo = 0, hi = nums.size() - 1;
3      while (lo <= hi) {
4          int mid = lo + (hi - lo) / 2;
5          if (nums[mid] == target) return mid;
6
7          // Determine which half is SORTED
8          if (nums[lo] <= nums[mid]) {
9              // Left half [lo..mid] is sorted
10             if (nums[lo] <= target && target < nums[mid]) {
11                 hi = mid - 1; // target in sorted left half
12             } else {
13                 lo = mid + 1; // target in right half
14             }
15         } else {
16             // Right half [mid..hi] is sorted
17             if (nums[mid] < target && target <= nums[hi]) {
18                 lo = mid + 1; // target in sorted right half
19             } else {
20                 hi = mid - 1; // target in left half
21             }
22         }
23     }
24     return -1;
25 }
26 // Time: O(log n)   Space: O(1)
27
28 // Example: nums=[4,5,6,7,0,1,2], target=0 → returns 4
29 // Example: nums=[4,5,6,7,0,1,2], target=3 → returns -1
```

5.3 Find Minimum in Rotated Sorted Array (LC 153) — Medium

The minimum element is the only one where $arr[i] < arr[i-1]$ (the inflection point). Strategy: the minimum is in the **unsorted half**. If $arr[mid] > arr[hi]$, the min is in the right half. Otherwise it's in the left (including mid).

C++

```
1  int findMin(vector<int>& nums) {
2      int lo = 0, hi = nums.size() - 1;
3      while (lo < hi) {
4          int mid = lo + (hi - lo) / 2;
5          if (nums[mid] > nums[hi]) {
6              // Minimum is in right half (mid is NOT the min)
7              lo = mid + 1;
8          } else {
9              // Minimum is in left half (mid COULD be the min)
10             hi = mid;
11         }
12     }
13     return nums[lo]; // lo == hi, single element = minimum
14 }
15 // Time: O(log n)   Space: O(1)
16
17 // Example: [3,4,5,1,2] → min is 1 (index 3)
18 // Step 1: lo=0,hi=4,mid=2  nums[2]=5 > nums[4]=2 → lo=3
19 // Step 2: lo=3,hi=4,mid=3  nums[3]=1 < nums[4]=2 → hi=3
20 // Step 3: lo==hi==3 → nums[3]=1
```

5.4 Search in Rotated Array II — with Duplicates (LC 81) — Medium

When duplicates exist, we can't determine which half is sorted when $\text{arr}[lo] == \text{arr}[mid] == \text{arr}[hi]$. Solution: shrink lo and hi by 1 each (linear worst case).

C++

```
1  bool search(vector<int>& nums, int target) {
2      int lo = 0, hi = nums.size() - 1;
3      while (lo <= hi) {
4          int mid = lo + (hi - lo) / 2;
5          if (nums[mid] == target) return true;
6
7          // Can't determine sorted half - shrink both boundaries
8          if (nums[lo] == nums[mid] && nums[mid] == nums[hi]) {
9              lo++; hi--;
10         } else if (nums[lo] <= nums[mid]) {
11             if (nums[lo] <= target && target < nums[mid])
12                 hi = mid - 1;
13             else
14                 lo = mid + 1;
15         } else {
16             if (nums[mid] < target && target <= nums[hi])
17                 lo = mid + 1;
18             else
19                 hi = mid - 1;
20         }
21     }
22     return false;
23 }
24 // Time: O(log n) avg, O(n) worst (all duplicates)   Space: O(1)
```

6 2D Matrix Binary Search

6.1 Search a 2D Matrix (LC 74) — Medium

The matrix has rows sorted left-to-right and each row's first element is greater than the last element of the previous row — effectively a flattened sorted array. **Key trick:** treat the $m \times n$ matrix as a 1D array of size $m \times n$ and map indices: $\text{row} = \text{mid} / n$, $\text{col} = \text{mid} \% n$.

```
C++
1  bool searchMatrix(vector<vector<int>>& matrix, int target) {
2      int m = matrix.size(), n = matrix[0].size();
3      int lo = 0, hi = m * n - 1;
4      while (lo <= hi) {
5          int mid = lo + (hi - lo) / 2;
6          int row = mid / n;          // map 1D index to 2D
7          int col = mid % n;
8          int val = matrix[row][col];
9          if (val == target) return true;
10         else if (val < target) lo = mid + 1;
11         else hi = mid - 1;
12     }
13     return false;
14 }
15 // Time: O(log(m*n)) = O(log m + log n)   Space: O(1)
16
17 // 4x4 matrix, target=7:
18 // [1, 3, 5, 7]
19 // [10, 11, 16, 20] → treated as [1,3,5,7,10,11,16,20,23,30,34,50]
20 // [23, 30, 34, 50] → mid=6 → row=1,col=2 → matrix[1][2]=16
```

6.2 Search a 2D Matrix II (LC 240) — Medium

Each row and column is sorted, but rows are NOT connected. Binary search on each row is $O(m \log n)$. Better: **start from top-right corner**. If $\text{val} > \text{target}$: move left (eliminate column). If $\text{val} < \text{target}$: move down (eliminate row). $O(m + n)$ — not binary search but elegant.

C++

```
1  bool searchMatrixII(vector<vector<int>>& matrix, int target) {
2      int m = matrix.size(), n = matrix[0].size();
3      int row = 0, col = n - 1;    // start: TOP-RIGHT corner
4      while (row < m && col >= 0) {
5          int val = matrix[row][col];
6          if (val == target) return true;
7          else if (val > target) col--; // too large → go left
8          else row++; // too small → go down
9      }
10     return false;
11 }
12 // Time: O(m + n)   Space: O(1)
13
14 // WHY top-right?
15 // Moving left DECREASES value (all left elements are smaller)
16 // Moving down INCREASES value (all below elements are larger)
17 // Every step eliminates a full row or column → at most m+n steps
```

6.3 Kth Smallest Element in Sorted Matrix (LC 378) — Medium

Binary search on the VALUE range [min, max]. For each candidate mid, count how many elements are $\leq$ mid using the top-right staircase walk ($O(m+n)$). Find the smallest value where count $\geq k$.

C++

```
1  int kthSmallest(vector<vector<int>>& matrix, int k) {
2      int n = matrix.size();
3      int lo = matrix[0][0], hi = matrix[n-1][n-1];
4
5      // Count elements <= mid using staircase from bottom-left
6      auto countLE = [&](int mid) {
7          int cnt = 0, row = n-1, col = 0;
8          while (row >= 0 && col < n) {
9              if (matrix[row][col] <= mid) {
10                 cnt += row + 1; // all elements above in this col are <= mid
11                 col++;
12             } else {
13                 row--;
14             }
15         }
16         return cnt;
17     };
18
19     while (lo < hi) {
20         int mid = lo + (hi - lo) / 2;
21         if (countLE(mid) < k) lo = mid + 1;
22         else hi = mid;
23     }
24     return lo; // smallest value with at least k elements <= it
25 }
26 // Time: O(n * log(max-min))   Space: O(1)
```

7 Binary Search on Answer (Parametric Search)

7.1 The Concept — Feasibility Function

Many optimization problems ask: 'what is the minimum X such that condition $C(X)$ is true?' If $C(X)$ is monotone (once true, remains true for all larger X), we can binary search on the **answer space** $[lo, hi]$ instead of the data. For each candidate answer mid , run a **feasibility check** in $O(n)$. Total: $O(n \log(\text{range}))$.

■ Binary Search on Answer Template

PATTERN: Binary Search on Answer

1. Identify: answer is some value X in a range $[lo, hi]$
2. Define: $\text{feasible}(X)$ — can we achieve the goal with X ?
3. Verify: $\text{feasible}()$ is MONOTONE — if $\text{feasible}(X)$ then $\text{feasible}(X+1)$
4. Binary search: find smallest X where $\text{feasible}(X) == \text{true}$
5. Template: `while(lo < hi){ mid = lo + (hi - lo) / 2; if(feasible(mid)) hi = mid; else lo = mid + 1; }`

7.2 Koko Eating Bananas (LC 875) — Medium

Guards return in h hours. Can eat at most k bananas/hour from one pile per hour. Find minimum k . **Binary search on $k \in [1, \max(\text{piles})]$** . Feasibility: can Koko finish all piles at speed k within h hours?

```
C++  
  
1  #include <cmath>  
2  
3  int minEatingSpeed(vector<int>& piles, int h) {  
4      int lo = 1, hi = *max_element(piles.begin(), piles.end());  
5  
6      // feasible(k): can Koko eat all piles at speed k in h hours?  
7      auto feasible = [&](int k) {  
8          long long hours = 0;  
9          for (int p : piles)  
10             hours += (p + k - 1) / k; // ceil(p/k) hours for pile p  
11          return hours <= h;  
12      };  
13  
14      while (lo < hi) {  
15          int mid = lo + (hi - lo) / 2;  
16          if (feasible(mid)) hi = mid; // mid works, try smaller  
17          else lo = mid + 1; // mid too slow, need faster  
18      }  
19      return lo; // smallest feasible speed  
20  }  
21  // Time: O(n * log(max_pile))  Space: O(1)
```

7.3 Capacity to Ship Packages (LC 1011) — Medium

Ship all packages in D days. Must ship in order. Binary search on capacity $\in [\max(\text{weight}), \text{sum}(\text{weights})]$. Feasibility: given capacity c, can we partition weights into D consecutive groups?

```
C++  
  
1  int shipWithinDays(vector<int>& weights, int days) {  
2      int lo = *max_element(weights.begin(), weights.end());  
3      int hi = 0; for (int w : weights) hi += w; // sum  
4  
5      auto feasible = [&](int cap) {  
6          int daysNeeded = 1, curLoad = 0;  
7          for (int w : weights) {  
8              if (curLoad + w > cap) {  
9                  daysNeeded++; // start new day  
10                 curLoad = 0;  
11             }  
12             curLoad += w;  
13         }  
14         return daysNeeded <= days;  
15     };  
16  
17     while (lo < hi) {  
18         int mid = lo + (hi - lo) / 2;  
19         if (feasible(mid)) hi = mid;  
20         else lo = mid + 1;  
21     }  
22     return lo;  
23 }  
24 // Time: O(n * log(sum-max))    Space: O(1)
```

7.4 Split Array Largest Sum (LC 410) — Hard

Split array into m subarrays to minimize the maximum subarray sum. Same feasibility function as capacity-to-ship but inverted: binary search on the allowed max sum, check if we can split into $\leq m$ parts.

C++

```
1  int splitArray(vector<int>& nums, int m) {
2      int lo = *max_element(nums.begin(), nums.end());
3      int hi = 0; for (int x : nums) hi += x;
4
5      auto feasible = [&](int limit) {
6          int parts = 1, curSum = 0;
7          for (int x : nums) {
8              if (curSum + x > limit) {
9                  parts++;
10                 curSum = 0;
11             }
12             curSum += x;
13         }
14         return parts <= m;
15     };
16
17     while (lo < hi) {
18         int mid = lo + (hi - lo) / 2;
19         if (feasible(mid)) hi = mid;
20         else lo = mid + 1;
21     }
22     return lo;
23 }
24 // Time: O(n * log(sum))   Space: O(1)
```

7.5 Minimum Number of Days to Make M Bouquets (LC 1482) — Medium

C++

```
1  int minDays(vector<int>& bloomDay, int m, int k) {
2      long long total = (long long)m * k;
3      if (total > bloomDay.size()) return -1; // impossible
4
5      int lo = 1, hi = *max_element(bloomDay.begin(), bloomDay.end());
6
7      auto feasible = [&](int day) {
8          int bouquets = 0, consecutive = 0;
9          for (int bd : bloomDay) {
10             if (bd <= day) {
11                 consecutive++;
12                 if (consecutive == k) { bouquets++; consecutive = 0; }
13             } else {
14                 consecutive = 0; // chain broken
15             }
16         }
17         return bouquets >= m;
18     };
19
20     while (lo < hi) {
21         int mid = lo + (hi - lo) / 2;
22         if (feasible(mid)) hi = mid;
23         else lo = mid + 1;
24     }
25     return lo;
26 }
27 // Time: O(n * log(max_day))   Space: O(1)
```

7.6 Minimized Maximum of Products Distributed (LC 2064) — Medium

C++

```
1  int minimizedMaximum(int n, vector<int>& quantities) {
2      int lo = 1, hi = *max_element(quantities.begin(), quantities.end());
3
4      // feasible: can we distribute all products to n stores
5      // such that max given to any store <= limit?
6      auto feasible = [&](int limit) {
7          int storesNeeded = 0;
8          for (int q : quantities)
9              storesNeeded += (q + limit - 1) / limit; // ceil(q/limit)
10         return storesNeeded <= n;
11     };
12
13     while (lo < hi) {
14         int mid = lo + (hi - lo) / 2;
15         if (feasible(mid)) hi = mid;
16         else lo = mid + 1;
17     }
18     return lo;
19 }
20 // Time: O(k * log(max_qty)) where k = quantities.size()
```

8 Peak Finding

8.1 Find Peak Element (LC 162) — Medium

A peak element is greater than its neighbors. Find ANY peak in $O(\log n)$. **Key insight:** if $\text{arr}[\text{mid}] < \text{arr}[\text{mid}+1]$, then a peak must exist in the right half (since values are climbing — there must be a local maximum to the right). Otherwise peak is in the left half (including mid).

```
C++  
  
1  int findPeakElement(vector<int>& nums) {  
2      int lo = 0, hi = nums.size() - 1;  
3      while (lo < hi) {  
4          int mid = lo + (hi - lo) / 2;  
5          if (nums[mid] < nums[mid + 1]) {  
6              lo = mid + 1;    // ascending slope → peak is to the right  
7          } else {  
8              hi = mid;        // descending slope → peak is here or left  
9          }  
10     }  
11     return lo;    // lo == hi = peak index  
12 }  
13 // Time:  $O(\log n)$     Space:  $O(1)$   
14  
15 // Why is this correct?  
16 //  $\text{nums}[-1] = \text{nums}[n] = -\text{infinity}$  (boundary condition)  
17 // If climbing right ( $\text{nums}[\text{mid}] < \text{nums}[\text{mid}+1]$ ):  
18 //   Either  $\text{nums}[\text{mid}+1]$  is a peak, or we continue climbing until we must descend  
19 //   Either way, a peak exists in  $[\text{mid}+1, \text{hi}]$ 
```

8.2 Find in Mountain Array (LC 1095) — Hard

A mountain array increases then decreases. Find the minimum index where $\text{arr}[i] == \text{target}$. Three steps: (1) find peak index, (2) binary search ascending left side, (3) binary search descending right side.

C++

```
1 // Mountain Array interface (simulate with vector for clarity)
2 int findInMountainArray(int target, vector<int>& arr) {
3     int n = arr.size();
4
5     // Step 1: Find peak index
6     int lo = 0, hi = n - 1;
7     while (lo < hi) {
8         int mid = lo + (hi - lo) / 2;
9         if (arr[mid] < arr[mid + 1]) lo = mid + 1;
10        else hi = mid;
11    }
12    int peak = lo;
13
14    // Step 2: Binary search on ascending left side [0..peak]
15    lo = 0; hi = peak;
16    while (lo <= hi) {
17        int mid = lo + (hi - lo) / 2;
18        if (arr[mid] == target) return mid;
19        else if (arr[mid] < target) lo = mid + 1;
20        else hi = mid - 1;
21    }
22
23    // Step 3: Binary search on descending right side [peak..n-1]
24    lo = peak; hi = n - 1;
25    while (lo <= hi) {
26        int mid = lo + (hi - lo) / 2;
27        if (arr[mid] == target) return mid;
28        else if (arr[mid] > target) lo = mid + 1; // descending: > means search right
29        else hi = mid - 1;
30    }
31    return -1;
32 }
33 // Time: O(log n) Space: O(1)
```

9 Advanced Variations

9.1 Sqrt(x) — Integer Square Root (LC 69) — Easy

Find $\text{floor}(\text{sqrt}(x))$ without using $\text{sqrt}()$. Binary search on answer in $[0, x]$. Find largest mid where $\text{mid} * \text{mid} \leq x$.

```
C++  
  
1  int mySqrt(int x) {  
2      if (x < 2) return x;  
3      int lo = 1, hi = x / 2;    // sqrt(x) <= x/2 for x >= 4  
4      while (lo < hi) {  
5          long long mid = lo + (hi - lo + 1) / 2; // RIGHT-biased (lo=mid branch)  
6          if (mid * mid <= x) lo = mid; // mid could be answer, keep  
7          else hi = mid - 1; // mid too large  
8      }  
9      return lo;  
10 }  
11 // Time: O(log x)   Space: O(1)  
12  
13 // Fractional sqrt with precision eps:  
14 double sqrtPrecise(double x, double eps = 1e-9) {  
15     double lo = 0, hi = max(1.0, x);  
16     while (hi - lo > eps) {  
17         double mid = lo + (hi - lo) / 2;  
18         if (mid * mid <= x) lo = mid;  
19         else hi = mid;  
20     }  
21     return lo;  
22 }  
23 // Time: O(log(x/eps))   Space: O(1)
```

9.2 Median of Two Sorted Arrays (LC 4) — Hard

Find median of two sorted arrays in $O(\log(\min(m,n)))$. **Key idea:** binary search on the **partition point** of the smaller array. A valid partition means: $\max(\text{left partition}) \leq \min(\text{right partition})$ for both arrays combined.

C++

```
1  double findMedianSortedArrays(vector<int>& A, vector<int>& B) {
2      if (A.size() > B.size()) return findMedianSortedArrays(B, A);
3      int m = A.size(), n = B.size();
4      int lo = 0, hi = m;
5
6      while (lo <= hi) {
7          int partA = lo + (hi - lo) / 2; // partition in A
8          int partB = (m + n + 1) / 2 - partA; // partition in B
9
10         // Elements just left/right of partition
11         int maxLeftA = (partA == 0) ? INT_MIN : A[partA - 1];
12         int minRightA = (partA == m) ? INT_MAX : A[partA];
13         int maxLeftB = (partB == 0) ? INT_MIN : B[partB - 1];
14         int minRightB = (partB == n) ? INT_MAX : B[partB];
15
16         if (maxLeftA <= minRightB && maxLeftB <= minRightA) {
17             // Valid partition found
18             if ((m + n) % 2 == 1)
19                 return max(maxLeftA, maxLeftB);
20             return (max(maxLeftA, maxLeftB) + min(minRightA, minRightB)) / 2.0;
21         } else if (maxLeftA > minRightB) {
22             hi = partA - 1; // too many elements from A on left
23         } else {
24             lo = partA + 1; // too few elements from A on left
25         }
26     }
27     return 0.0; // unreachable if inputs are valid
28 }
29 // Time: O(log(min(m,n))) Space: O(1)
```

9.3 Find Minimum in Rotated Array II (LC 154) — Hard

With duplicates. Same idea as LC 153 but when `arr[mid] == arr[hi]`, we can't determine which half has the minimum — decrement `hi` by 1.

C++

```
1  int findMin(vector<int>& nums) {
2      int lo = 0, hi = nums.size() - 1;
3      while (lo < hi) {
4          int mid = lo + (hi - lo) / 2;
5          if (nums[mid] > nums[hi]) {
6              lo = mid + 1;           // min is in right half
7          } else if (nums[mid] < nums[hi]) {
8              hi = mid;              // min is in left half (including mid)
9          } else {
10             hi--;                  // ambiguous: shrink hi by 1 safely
11         }
12     }
13     return nums[lo];
14 }
15 // Time: O(log n) avg, O(n) worst   Space: O(1)
```

9.4 Time Based Key-Value Store (LC 981) — Medium

Store multiple values per key with timestamps. Get the value with the largest timestamp $\leq$ given time. Binary search on timestamps stored per key.

C++

```
1  #include <unordered_map>
2
3  class TimeMap {
4      unordered_map<string, vector<pair<int, string>>> store;
5      // key → [(timestamp, value), ...] sorted by timestamp
6  public:
7      void set(string key, string value, int timestamp) {
8          store[key].push_back({timestamp, value});
9          // timestamps always increase per problem guarantee
10     }
11     string get(string key, int timestamp) {
12         if (!store.count(key)) return "";
13         auto& v = store[key];
14         // Binary search: find last ts <= timestamp
15         int lo = 0, hi = v.size() - 1, ans = -1;
16         while (lo <= hi) {
17             int mid = lo + (hi - lo) / 2;
18             if (v[mid].first <= timestamp) {
19                 ans = mid;           // valid, try further right
20                 lo = mid + 1;
21             } else {
22                 hi = mid - 1;
23             }
24         }
25         return ans == -1 ? "" : v[ans].second;
26     }
27 };
28 // set: O(1)   get: O(log n)   Space: O(total_sets)
```

10 Complexity & Pattern Cheat Sheet

10.1 All Templates Side-by-Side

C++

```

1  // ■■■■ TEMPLATE 1: Exact Match (lo <= hi) ■■■■
2  int lo=0, hi=n-1;
3  while (lo <= hi) {
4      int mid = lo + (hi-lo)/2;
5      if (arr[mid] == target) return mid;
6      else if (arr[mid] < target) lo = mid+1;
7      else hi = mid-1;
8  } // returns -1 after loop
9
10 // ■■■■ TEMPLATE 2: Lower Bound (lo < hi) ■■■■
11 int lo=0, hi=n; // hi=n (past end)
12 while (lo < hi) {
13     int mid = lo + (hi-lo)/2;
14     if (arr[mid] < target) lo = mid+1; // strict < for lower bound
15     else hi = mid;
16 } // lo = first index where arr[i] >= target
17
18 // ■■■■ TEMPLATE 2: Upper Bound (lo < hi) ■■■■
19 int lo=0, hi=n;
20 while (lo < hi) {
21     int mid = lo + (hi-lo)/2;
22     if (arr[mid] <= target) lo = mid+1; // <= for upper bound
23     else hi = mid;
24 } // lo = first index where arr[i] > target
25
26 // ■■■■ TEMPLATE 3: Search on Answer (minimize) ■■■■
27 int lo=MIN_ANSWER, hi=MAX_ANSWER;
28 while (lo < hi) {
29     int mid = lo + (hi-lo)/2;
30     if (feasible(mid)) hi = mid; // mid works, find smaller
31     else lo = mid+1; // mid doesn't work, go larger
32 } // lo = minimum feasible answer
33
34 // ■■■■ TEMPLATE 3: Search on Answer (maximize) ■■■■
35 while (lo < hi) {
36     int mid = lo + (hi-lo+1)/2; // RIGHT-biased mid
37     if (feasible(mid)) lo = mid; // mid works, try larger
38     else hi = mid-1; // mid doesn't work
39 } // lo = maximum feasible answer

```

10.2 Complete Complexity Table

Variation	Time	Space	Key Condition
Classic binary search	$O(\log n)$	$O(1)$	Sorted array, exact match
Lower/Upper bound	$O(\log n)$	$O(1)$	Sorted array, range query
Recursive binary search	$O(\log n)$	$O(\log n)$	Stack frames
Search in rotated array	$O(\log n)$	$O(1)$	One half always sorted
Find min in rotated	$O(\log n)$	$O(1)$	Compare mid to hi
2D matrix (flatten)	$O(\log(mn))$	$O(1)$	Each row > prev row last
2D matrix II (staircase)	$O(m+n)$	$O(1)$	Row+col sorted (not BS!)
Search on answer	$O(n * \log(\text{range}))$	$O(1)$	Monotone feasibility
Peak finding	$O(\log n)$	$O(1)$	Slope direction tells side
Median two sorted arrays	$O(\log \min(m,n))$	$O(1)$	Binary search on partition
Fractional / real-valued BS	$O(\log(\text{range}/\epsilon))$	$O(1)$	Continuous search space
Kth smallest in matrix	$O(n \log(\text{max-min}))$	$O(1)$	BS on value range

Table 4: Complete complexity table for all binary search variations

10.3 Decision Tree — Which Template?

C++

```

1  /*
2  * PROBLEM TYPE                                → TEMPLATE
3  *
4  * 'Is target in sorted array?'                → Template 1 (lo<=hi, exact)
5  * 'First index where arr[i] >= target'        → Template 2 lower_bound
6  * 'Last index where arr[i] <= target'         → Template 2 upper_bound-1
7  * 'Count occurrences of target'              → upper_bound - lower_bound
8  * 'Search in rotated sorted array'           → Template 1 + sorted-half check
9  * 'Minimum in rotated array'                 → Template 2 + compare mid to hi
10 * 'Find peak element'                        → Template 2 + slope direction
11 * 'Minimise the maximum / minimum X'          → Template 3 + feasible(mid)
12 * 'Can we complete in X days/speed/etc.'      → Template 3 (monotone condition)
13 * 'Search in 2D matrix (rows connected)'      → Flatten to 1D, Template 1
14 * 'Search in 2D matrix (rows independent)'    → Staircase O(m+n), NOT binary search
15 * 'Median of two sorted arrays'              → Binary search on partition
16 * 'Integer square root'                      → Template 3 maximise version
17 * 'Find in mountain array'                   → 3-step: peak + 2x binary search
18 */

```

10.4 Complete LeetCode Problem Map

#	Problem	Template	Diff
33	Search in Rotated Sorted Array	T1 + sorted half	Medium
34	First and Last Position in Sorted Array	T2 lower+upper bound	Medium
35	Search Insert Position	T2 lower bound	Easy
69	Sqrt(x)	T3 maximize answer	Easy
74	Search a 2D Matrix	T1 flatten to 1D	Medium
81	Search in Rotated Array II (duplicates)	T1 + shrink on dup	Medium
153	Find Minimum in Rotated Sorted Array	T2 compare mid to hi	Medium
154	Find Minimum in Rotated Array II (dup)	T2 + hi-- on ambiguity	Hard
162	Find Peak Element	T2 slope direction	Medium
240	Search a 2D Matrix II	Staircase O(m+n)	Medium
275	H-Index II	T2 lower bound	Medium
278	First Bad Version	T2 lower bound	Easy
374	Guess Number Higher or Lower	T1 exact match	Easy
378	Kth Smallest in Sorted Matrix	T3 on value range	Medium
410	Split Array Largest Sum	T3 minimize max	Hard
704	Binary Search	T1 classic	Easy
744	Find Smallest Letter Greater Than Target	T2 upper bound	Easy
875	Koko Eating Bananas	T3 minimize speed	Medium
981	Time Based Key-Value Store	T1 last <= timestamp	Medium
1011	Capacity to Ship Packages	T3 minimize capacity	Medium
1095	Find in Mountain Array	T2 peak + 2x search	Hard
1231	Divide Chocolate	T3 maximize min piece	Hard
1283	Find Smallest Divisor	T3 minimize divisor	Medium
1482	Minimum Days to Make m Bouquets	T3 minimize days	Medium
1870	Minimum Speed to Arrive on Time	T3 minimize speed	Medium
2064	Minimized Maximum Products Distributed	T3 minimize max	Medium
4	Median of Two Sorted Arrays	T1 on partition point	Hard

Table 5: Complete LeetCode problem map — 26 problems across all binary search templates

■ Golden Rules — Binary Search

1. First question: Is the search space MONOTONE? If not, binary search doesn't apply.
2. Always use $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$ — never $(\text{lo} + \text{hi}) / 2$. Overflow kills production code.
3. Template 1 ($\text{lo} \leq \text{hi}$): for exact match. Template 2 ($\text{lo} < \text{hi}$): for bound finding and answer search.
4. $\text{lo} = \text{mid} + 1$ always valid (excludes mid). $\text{hi} = \text{mid}$ only when mid could still be the answer.
5. RIGHT-biased mid ($\text{lo} + (\text{hi} - \text{lo} + 1) / 2$) required whenever your branch writes $\text{lo} = \text{mid}$ (not $\text{lo} = \text{mid} + 1$).
6. For 'search on answer': $\text{lo} = \text{worst_case}$, $\text{hi} = \text{best_case}$. `feasible()` must run in $O(n)$.
7. Rotated arrays: ONE half is ALWAYS fully sorted — use this to decide which half to recurse on.
8. When problem says 'minimum X such that condition holds' → binary search on X, condition is `feasible()`.
9. 2D matrix search: if rows connected (ascending) → flatten to 1D $O(\log mn)$. Else → staircase $O(m+n)$.
10. After writing binary search, always verify: does it terminate? Does it give the right answer for $n=1$?